



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Edwin David Santisteban Grajeda

Carné: 1047425

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
Es uno de los tipos de arquitectura de diseño de software que se utiliza para crear servicios web que permiten el intercambio de datos entre diferentes sistemas
2. ¿Qué es un endpoint?
Es una ubicación específica dentro de una API, en donde se harán intercambio de dato u acciones
3. ¿Qué diferencia existe entre una URL y un endpoint?
Un endpoint es una acción específica dentro de un URL, mientras que un URL es la dirección completa usada para acceder a un endpoint específico
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
Get sirve para obtener un valor específico, put/patch nos ayuda a actualizar, post nos ayuda a añadir o crear, y delete elimina.
5. ¿Qué es JSON y por qué es común en las APIs?
Java Script Object Notation Es común en las APIs porque es fácil de leer para las personas y de interpretar para las máquinas, es liviano, lo que reduce el tamaño de las transferencias, es independiente del lenguaje: Java, Python, JavaScript, C#, etc. lo soportan y se integra de forma natural con JavaScript y con servicios web REST.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
Es un número de tres dígitos que el servidor incluye en su respuesta para indicar si la solicitud tuvo éxito, falló o requiere otra acción, como el **200 OK**: la solicitud se procesó correctamente y se devuelve el resultado, **404 Not Found**: el recurso solicitado no existe (por ejemplo, una URL o un ID inexistente) y **500 Internal Server Error**: ocurrió un error inesperado en el servidor al procesar la solicitud.
7. ¿Qué información se puede encontrar en la respuesta de una API?
Código de estado: indica el resultado (200, 404, 500...).
Encabezados (headers): metadatos como Content-Type, Content-Length, fecha, cookies o información de caché.
Cuerpo (body): los datos solicitados, normalmente en JSON o XML, o un mensaje de error.
Tiempo de respuesta y tamaño: útiles para medir el rendimiento.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
Permite probar los endpoints sin escribir código cliente, soporta todos los métodos (GET, POST, PUT, DELETE, etc.), con encabezados, parámetros y cuerpo personalizados, muestra claramente la respuesta: estado, headers, cuerpo y tiempo, facilita la autenticación (tokens, API keys, OAuth).
9. ¿Qué es y para qué sirve Swagger UI?
Es una herramienta que genera una interfaz web interactiva a partir de la documentación de una API escrita con la especificación OpenAPI, generalmente se usa para visualizar todos los endpoints, parámetros, modelos de datos y respuestas posibles, probar los endpoints directamente desde el navegador y mantener la documentación siempre actualizada y fácil de consultar para desarrolladores y clientes.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
Postman es una plataforma para diseñar, probar, documentar y colaborar en APIs. Su función es enviar solicitudes HTTP a un servicio y analizar sus respuestas

1. Preparación del entorno



2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
/pet/ID
- Indique el método HTTP utilizado.
Get
- Indique qué parámetro necesita la solicitud.
PetID
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
Codigo 200 ya que devuelve con éxito la mascota
- Explique con sus palabras qué información contiene la respuesta.
Obtiene el nombre de la mascota y nombre

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
El método put porque es el que actualiza la información
- ¿Qué datos fueron enviados en el body?
Los datos que se modificaron sumados a los ya existentes no modificados
- ¿Qué código HTTP devolvió la API?
Un código 200 porque fue válido el intercambio
- ¿Cómo comprobó que el cambio realmente ocurrió?
Se usó el método get para comprobar que si se cambió

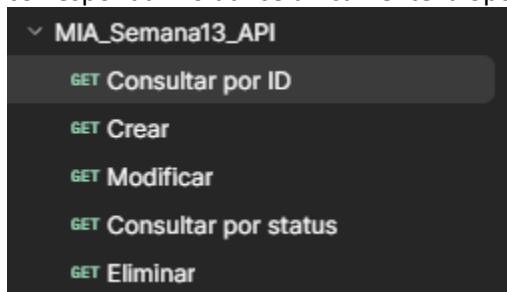


5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
Se usó el parámetro de "available" o disponible.
- Explique dónde se envía ese parámetro.
Se envía en el curl y el request
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.
Me saldrán solo los animales cuyo valor de estado sea pending

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.



7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa



ParametersCancel

Name	Description
petid * required integer(\$int64) (path)	ID of pet to return

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  https://petstore3.swagger.io/api/v3/pet/-12 \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/-12
```

Server response

Code	Details
404	Error: response status is 404 Response body <pre>can't parse JSON. Raw result: Pet not found</pre> Download

Se hizo un get por ID, se obtuvo un code 404 de no encontrado, la respuesta fue que no se encontró, debería haber enviado información de regreso de la mascota buscada, al ingresar mal el parámetro la respuesta fue que no encontró coincidencias.

Code	Details
400	Error: response status is 400 Response body <pre>{ "code": 400, "message": "Input error: unable to convert input to io.swagger.petstore.model.Pet" }</pre> Download Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 94 content-type: application/json date: Sat,03 Oct 2026 03:57:59 GMT server: Jetty(9.4.53.v20231009)</pre>

Responses

Se hizo un post, se obtuvo el error 400 por invalid input, la respuesta fue que un parámetro no se pudo convertir para poder realizar el post, debería haber creado una nueva mascota con los datos ingresados, al probar hacer post daba error porque hacía falta el ID de la mascota



Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1012,
    "name": "doggie",
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code	Details
404	Error: response status is 404

Response body

```
can't parse JSON. Raw result:
Pet not found
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 13
content-type: application/json
date: Sat,03 Oct 2026 04:01:32 GMT
server: Jetty(9.4.53.v20231009)
```

Se hizo un put, se obtuvo el error 404 por invalid input, la respuesta fue que la mascota no fue encontrada, debería haber actualizado a la mascota con el mismo ID que se ingresó, al probar hacer el put no encontraba la mascota con el ID ingresado por lo que no actualizaba nada.

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Para Registrar se usa Post (endpoint /pet), para consultar se usa Get (endpoint /pet/ID), para actualizar estado se usa Put (enpoint /pet), para eliminar se usa Delete (enpoint /pet/ID) y para volver a consultar si existe se usa Get (endpoint /pet/ID).

Debe analizar si la respuesta es 200 para los primeros 4 métodos, post, get, put, delete, (todo correcto) o si es 404 (no encontrado) en el último método.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.



The screenshot displays a web browser with the Swagger UI for the Petstore API. The API is titled "pet" and "store". The "pet" section lists several endpoints: PUT /pet (Update an existing pet), POST /pet (Add a new pet to the store), GET /pet/findByStatus (Finds Pets by status), GET /pet/findByTags (Finds Pets by tags), GET /pet/{petId} (Find pet by ID), POST /pet/{petId} (Updates a pet in the store with form data), DELETE /pet/{petId} (Deletes a pet), and POST /pet/{petId}/uploadImage (Uploads an image). The "store" section lists: GET /store/inventory (Returns pet inventories by status), POST /store/order (Place an order for a pet), and GET /store/order/{orderId} (Find purchase order by ID).

Below the Swagger UI, a terminal window shows a curl command and its response. The curl command is:

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 104742,
    "name": "Sebas",
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }'
```

The terminal output shows a 500 status code error:

```
500 Error: response status is 500
Response body
{
  "code": 500,
  "message": "There was an error processing your request. It has been logged (ID: 8d9c1231bd8b5c1f)"
}
Response headers
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 110
content-type: application/json
date: Wed, 30 Sep 2026 13:48:01 GMT
server: Letty(9.4.53.v20231009)
```

Creación de una nueva mascota (método post):



Request body required

application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 10474,
  "name": "chucho",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Execute

Clear

Code Details

200

Response body

```
{
  "id": 10474,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "chucho",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```



Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origins: *
access-control-expose-headers: Content-Disposition
content-length: 140
content-type: application/json
date: Sat,03 Oct 2026 03:08:14 GMT
server: Jetty(9.4.53.v20231009)
```

Uso de método get (por medio de ID) para obtener una mascota en específico

Parameters

Cancel

Name Description

petId * required ID of pet to return

integer(\$int64)
(path)

10474

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/10474' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/10474



Code

200

Response body

```
{
  "id": 10474,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "chucho",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 140
content-type: application/json
date: Sat,03 Oct 2026 03:10:35 GMT
server: Jetty(9.4.53.v20231009)
```

Download

Método put para modificar la información de una mascota ya existente

PUT /pet Update an existing pet.

Update an existing pet by Id.

Parameters

No parameters

Request body required

application/json

Update an existent pet in the store

Edit Value | Schema

```
{
  "id": 10474,
  "name": "snoopy",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Uso de método get para ver la mascota modificada



Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/10474' \
  -H 'accept: application/json'
```



Request URL

```
https://petstore3.swagger.io/api/v3/pet/10474
```

Server response

Code Details

200

Response body

```
{
  "id": 10474,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "snoopy",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```



Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 140
content-type: application/json
date: Sat,03 Oct 2026 03:17:06 GMT
server: Jetty(9.4.53.v20231009)
```